

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6-8

Student Name: D. Vijaya Sri

1. TITLE

Middleware in Express.js — Request Logging

2. AIM

To understand middleware in Express.js and develop a custom middleware that logs incoming HTTP requests.

3. OBJECTIVE

The objectives of this experiment are:

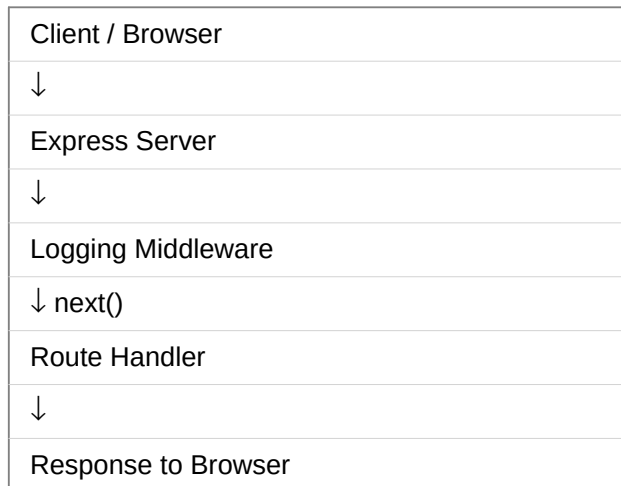
- To understand the concept of middleware in Express.js.
- To understand the request-response cycle.
- To create and register custom middleware.
- To log the HTTP method and requested URL of incoming requests.
- To use **next()** to pass control to the route handler.
- To verify the middleware using a browser and the terminal.

4. THEORY

Middleware in Express.js is a function that has access to the request object, response object, and the **next()** function. It can execute code, modify the request or response, end the request-response cycle, or pass control to the next middleware or route.

Middleware is commonly registered using **app.use()**. In this experiment, a custom logging middleware records the current time, HTTP method, and requested URL for every incoming request.

Request-response cycle:



5. ALGORITHM / PROCEDURE

1. Create a new Node.js project folder.
2. Initialize the project using **npm init -y**.
3. Install Express using **npm install express**.
4. Import the Express module into **app.js**.
5. Create an Express application.
6. Define a custom **requestLogger** middleware function.
7. Log the current time, HTTP method, and requested URL.
8. Call **next()** so that the request continues to the route.
9. Create a GET route for the root URL.
10. Start the Express server on port 3000.
11. Open **http://localhost:3000/** in a browser.
12. Observe the webpage response and the request log in the terminal.
13. Refresh the page and verify that another GET request is logged.

6. PROGRAM

app.js

```
const express = require("express");

const app = express();

const PORT = 3000;

// Custom logging middleware
function requestLogger(req, res, next) {
  const currentTime = new Date().toLocaleString();

  console.log(
    `[${currentTime}] ${req.method} ${req.url}`
  );

  // Pass control to the next middleware or route
  next();
}

// Register the middleware
app.use(requestLogger);

// GET route
app.get("/", (req, res) => {
  res.send(`
    <h1>Express.js Middleware Demo</h1>
    <p>Request logging middleware is working.</p>
  `);
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Project structure

```
Q8-Express-Middleware/
├──
├── app.js
├── package.json
└── package-lock.json
```

The **node_modules** directory is generated by NPM after installing Express and normally should not be uploaded to GitHub.

7. CODE EXPLANATION

Code / Method	Explanation
<code>require("express")</code>	Imports the Express.js package.
<code>express()</code>	Creates an Express application.
<code>requestLogger(req, res, next)</code>	Defines custom middleware with access to the request, response, and next function.
<code>new Date().toLocaleString()</code>	Gets the current local date and time for the log.
<code>req.method</code>	Provides the HTTP method used by the incoming request.
<code>req.url</code>	Provides the requested URL.
<code>next()</code>	Passes control to the next middleware or route handler.
<code>app.use(requestLogger)</code>	Registers the logging middleware for incoming requests.
<code>app.get("/")</code>	Handles GET requests to the root URL.
<code>res.send()</code>	Sends the HTML response to the browser.
<code>app.listen()</code>	Starts the Express server on port 3000.

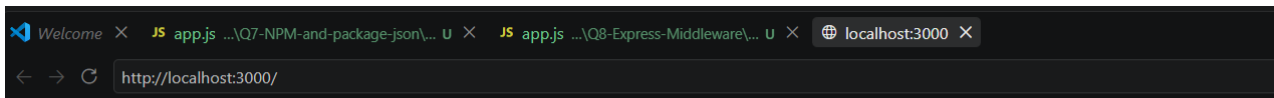
8. TEST CASES AND EXECUTION DATA

The Express application was executed and tested using the browser and terminal. The actual screenshot records the installation, server startup, browser request, favicon request, and page refresh request.

Test Case	Test / Input	Expected Result	Actual Result	Status
TC-01	Run npm install express	Express should install without errors.	68 packages added; 0 vulnerabilities found.	Pass
TC-02	Run node app.js	Server should start on port 3000.	Server running at http://localhost:3000	Pass
TC-03	Open http://localhost:3000/	Middleware should log GET / and browser should display the demo.	GET / logged and webpage displayed correctly.	Pass
TC-04	Browser requests favicon	Middleware should log the request.	GET /favicon.ico logged.	Pass
TC-05	Refresh root page	A new GET / request should be logged.	New GET / logged successfully.	Pass

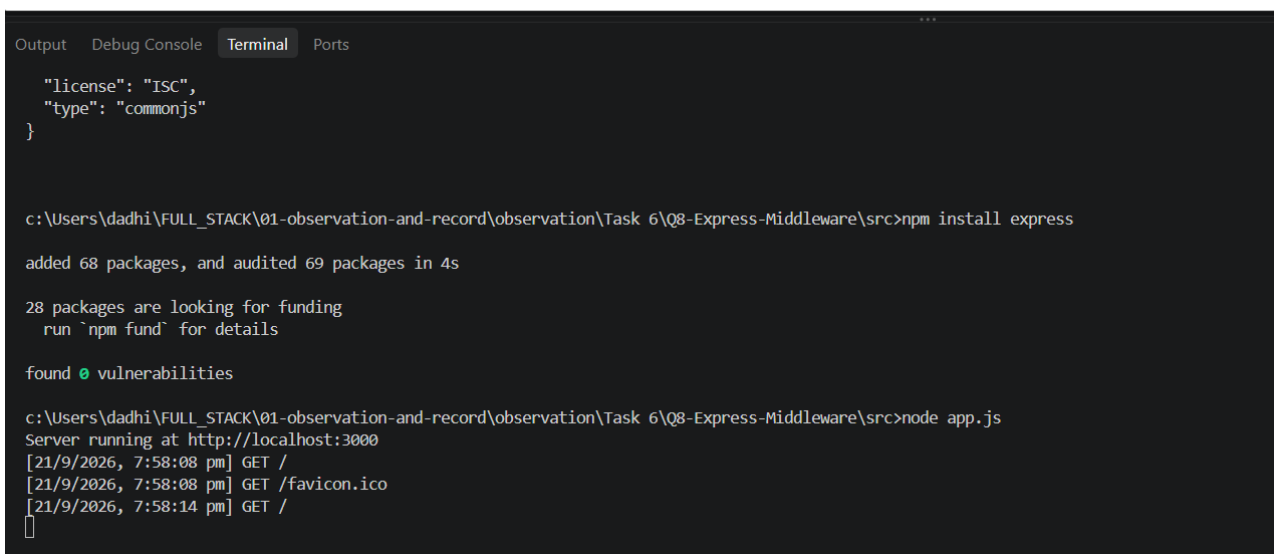
9. OUTPUT

Output 1: Browser and Terminal Execution



Express.js Middleware Demo

Request logging middleware is working.



9. OUTPUT

Observed execution results

Item	Observed Value
Express installation	68 packages added; 69 packages audited; 0 vulnerabilities found.
Server	Server running at http://localhost:3000
Browser output	Express.js Middleware Demo — Request logging middleware is working.
Logged request 1	GET /favicon.ico at 7:58:08 pm
Logged request 2	GET / at 7:58:14 pm

10. RESULT

Thus, middleware in Express.js was successfully demonstrated by developing a custom request-logging middleware. The middleware logged incoming HTTP requests, used **next()** to continue the request-response cycle, and the application was successfully verified through the browser and terminal using the actual execution output.